Campus Sports Tournament Management System

Comprehensive DBMS Guide: SQL Joins, Aggregate Functions, CTEs & Relational Mechanics

> **Document Type:** College DBMS Lab Project Architectural & Oral Defense Handout

> **Database Engine:** MySQL 8.0 Community Edition (InnoDB, `utf8mb4`)

> **Artifact Produced:** `docs/DBMS_QUERIES_AND_AGGREGATES.pdf` (Derived from this markdown specification)

> **Target Audience:** DBMS Lab Instructors, Oral Defense Examiners, and Project Collaborators

---

Table of Contents

1. Executive Overview & Query Architecture
2. Theoretical Foundations of Relational Joins

- 2.1 Relational Algebra Join Formalism
- 2.2 MySQL 8.0 Join Execution Algorithms (Nested Loop & Hash Join)
- 2.3 INNER JOIN vs LEFT OUTER JOIN: The "Zero-Match Disappearance" Principle
- 2.4 Dual Foreign Key Self-Referencing Joins (Home vs Away Teams)
- 2.5 Exclusive-Arc / Polymorphic Joins via Set Union

1. Theoretical Foundations of SQL Aggregates & Grouping

- 3.1 The Relational Aggregation Operator (γ)
- 3.2 The Crucial Difference: `COUNT(*)` vs `COUNT(column_name)`
- 3.3 Grouping Semantics & The `ONLY_FULL_GROUP_BY` Standard
- 3.4 Filter Lifecycle: `WHERE` vs `HAVING`
- 3.5 Boolean Expression Aggregation (`SUM(condition)`)
- 3.6 Type Safety: Unsigned Integer Underflow & `CAST(... AS SIGNED)`
- 3.7 Null Coalescing with `COALESCE()`

1. Advanced Relational Constructs: CTEs & Window Functions

- 4.1 Common Table Expressions (`WITH ... AS`)
- 4.2 Window Functions vs Standard Aggregation
- 4.3 Comparative Ranking: `DENSE_RANK()` vs `RANK()` vs `ROW_NUMBER()`

1. Deep-Dive Walkthrough of Flagship Production Queries

- 5.1 Deep Dive: `v_points_table` (Multi-CTE Standings Engine)
- 5.2 Deep Dive: `v_medal_tally` (Polymorphic Exclusive-Arc Rollup)

- 5.3 Deep Dive: `v_match_schedule` (6-Table Operations Schedule)
- 5.4 Deep Dive: `fn_team_points` (Deterministic Score Function)
- 5.5 Deep Dive: `sp_generate_group_fixtures` & `sp_advance_knockout`

1. Complete Master Query Catalog Across the Project

- 6.1 Database Views (`sql/views.sql`)
- 6.2 Stored Routines (`sql/procedures.sql`)
- 6.3 Named Demonstration Queries (`sql/queries.sql`)
- 6.4 Integrity Triggers (`sql/triggers.sql`)
- 6.5 Node.js Application Layer Queries (`app/server.js`)

1. DBMS Lab Viva Defense & Oral Examination Cheat-Sheet

---

1. Executive Overview & Query Architecture

In relational database management systems, data is normalized into discrete, decoupled tables to eliminate redundancy, avoid update anomalies, and enforce Boyce-Codd Normal Form (BCNF). However, real-world business questions—such as "*Which department has the most medals?*", "*What are the current football group standings?*", or "*Who is scheduled to play on Main Turf at 10:00 AM?*"—require assembling records from multiple relations and computing mathematical summaries.

Our **Campus Sports Tournament Management System** deliberately implements this computation **inside the database engine** rather than the client application.



By pushing joins, aggregations, and windowing into MySQL:

1. **Network traffic is minimized:** Instead of transmitting hundreds of raw match records to Node.js and calculating points in JavaScript, MySQL returns the exact 4-row points table.
2. **ACID transactions and concurrency are guaranteed:** Database locks and snapshot isolation guarantee consistent reads without race conditions.
3. **Application code remains thin and declarative:** The Node.js application simply executes `SELECT * FROM v_points_table`.

---

2. Theoretical Foundations of Relational Joins

2.1 Relational Algebra Join Formalism

In formal relational algebra, a join is a derived operation combining the Cartesian product ($\times$) of two relations with a selection (σ) based on a join condition:

$$R \bowtie_\theta S = \sigma_\theta (R \times S)$$

When the condition θ uses exclusively equality predicates between foreign keys and primary keys ($R.A = S.B$), the operation is an **Equi-Join**.

In our database, every join links child foreign keys to parent primary keys. For example:

- `event.sport_id = sport.sport_id`
- `match.venue_id = venue.venue_id`
- `registration.event_id = event.event_id`

2.2 MySQL 8.0 Join Execution Algorithms

When MySQL 8.0 executes a query involving joins, the InnoDB storage engine and query optimizer employ specific execution strategies:

1. **Index Nested-Loop Join (INLJ):**

When the join predicate matches an index (such as a Primary Key or a Foreign Key index), MySQL iterates through the outer table and performs instant $O(\log N)$ B-tree lookups on the inner table. This is the primary algorithm used across our schema because all foreign keys are indexed.

1. **Hash Join (MySQL 8.0.18+):**

If no usable index exists for an equi-join, MySQL builds an in-memory hash table of the smaller relation on the join attribute, then scans the larger relation to find matches in $O(M + N)$ linear time.

1. **Block Nested-Loop (BNL):**

Used when no index exists and hash join is not applicable; loads chunks of the outer relation into the `join_buffer_size` cache to reduce disk reads.

2.3 INNER JOIN vs LEFT OUTER JOIN: The "Zero-Match Disappearance" Principle

The fundamental distinction between an `INNER JOIN` and a `LEFT OUTER JOIN` is row preservation:

- `INNER JOIN` **($R \bowtie S$):** Emits a resulting tuple if and only if **both** relations satisfy the join condition. If an entity in R has no matching child in S, that entity is **discarded**.
- `LEFT OUTER JOIN` **($R \mathbin{⟕} S$):** Emits **all** tuples from the left relation R. If no matching tuple exists in S, the columns of S are populated with `NULL`.

Why This Distinction is Vital in Our Project

Consider generating the tournament points table:

- If we use an `INNER JOIN` between `team` and `match`, **any team that has played zero matches will be completely eliminated from the standings!**
- In a tournament, when a group stage is created, all registered teams must immediately appear on the leaderboard with:
- `Played = 0`
- `Won = 0`
- `Drawn = 0`
- `Lost = 0`
- `Points = 0`

To achieve this, `v_points_table` seeds the registered teams first, and then performs a `LEFT JOIN` onto completed match results. If matches have not taken place, the match columns are `NULL`, which we coalesce to `0`.

2.4 Dual Foreign Key Self-Referencing Joins (Home vs Away Teams)

A single table can reference another table multiple times for distinct semantic roles. In `match`:

- `home_team_id` references `team.team_id`
- `away_team_id` references `team.team_id`

To retrieve the readable names of both opposing teams in a single SQL query, the query must join the `team` table **twice**, using unique table aliases:

```
SELECT
    m.match_id,
    home.name AS home_team,
    away.name AS away_team
FROM `match` AS m
JOIN team AS home ON home.team_id = m.home_team_id
JOIN team AS away ON away.team_id = m.away_team_id;
```

Relational Execution Mechanics:

1. The engine scans `match`.
2. For each match tuple, it evaluates `home.team_id = m.home_team_id` against the B-tree index of `team` to extract the home team name.
3. In parallel, it evaluates `away.team_id = m.away_team_id` against the same index to extract the away team name.
4. The database check constraint `ck_match_distinct_teams (home_team_id <> away_team_id)` guarantees that `home` and `away` can never represent the same department team in a match.

2.5 Exclusive-Arc / Polymorphic Joins via Set Union

In sports management, some events are played by **teams** (e.g. Football, Basketball), while other events are contested by **individual athletes** (e.g. Athletics 100m Sprint).

The `award` table enforces an Exclusive ARC constraint:

```
CONSTRAINT ck_award_exactly_one_recipient
CHECK ((student_id IS NOT NULL) <> (team_id IS NOT NULL))
```

Every award is given to **either** a team **or** an individual student, never both.

To calculate the overall **Department Medal Tally**, the database must map every award back to its sponsoring department:

- Team awards: `award` $\rightarrow$ `team` $\rightarrow$ `department`
- Student awards: `award` $\rightarrow$ `student` $\rightarrow$ `department`

We resolve this polymorphic relationship inside `v_medal_tally` using a **Common Table Expression (CTE)** combined with `UNION ALL`:

```
WITH award_departments AS (
    -- Path 1: Team awards
    SELECT d.department_id, d.name AS department_name, a.award_type
    FROM award AS a
    JOIN team AS t ON t.team_id = a.team_id
    JOIN department AS d ON d.department_id = t.department_id

    UNION ALL

    -- Path 2: Individual student awards
    SELECT d.department_id, d.name AS department_name, a.award_type
    FROM award AS a
    JOIN student AS s ON s.student_id = a.student_id
    JOIN department AS d ON d.department_id = s.department_id
)
```

`UNION ALL` concatenates the tuple streams from both relational paths without the costly de-duplication overhead of `UNION`, creating a unified relation ready for aggregation.

---

3. Theoretical Foundations of SQL Aggregates & Grouping

3.1 The Relational Aggregation Operator (γ)

In formal relational algebra, aggregation is denoted by the calligraphic gamma operator:

$$\gamma_{G, F_1(A_1), F_2(A_2), \dots, F_n(A_n)} (R)$$

Where:

- G is the list of grouping attributes (`GROUP BY`).
- F_i is an aggregate function (`COUNT` , `SUM` , `AVG` , `MIN` , `MAX`).
- A_i is the attribute being aggregated.

Aggregation partitions the input relation R into subsets of tuples having identical values for attributes G, and reduces each partition to a single summary tuple.

3.2 The Crucial Difference: `COUNT(*)` vs `COUNT(column_name)`

A frequent trap in DBMS design is assuming `COUNT(*)` and `COUNT(column)` behave identically. **They do not:**

`COUNT(*)`	**Counts the total number of rows in the partition.**	**Counts the row even if all joined columns are `NULL` .**
`COUNT(expression)`	Evaluates `expression` for each row in the partition.	**Ignores and skips `NULL` values.**

Practical Example in Our Database:

In `v_points_table` , we compute matches played:

```
COUNT(tr.team_id) AS played
```

- If a team has played zero matches, the `LEFT JOIN` produces a row where `tr.team_id IS NULL`.
- If we had used `COUNT(*)`, MySQL would count the row itself and report **1 match played** for an inactive team!
- Because we used `COUNT(tr.team_id)`, MySQL encounters `NULL`, ignores it, and correctly returns **0 matches played**!

Similarly, in Query `Q04` (Squad Sizes):

```
SELECT t.team_id, t.name, COUNT(tm.student_id) AS squad_size
FROM team AS t
LEFT JOIN team_member AS tm ON tm.team_id = t.team_id
GROUP BY t.team_id, t.name;
```

If a team has no registered members yet, `COUNT(tm.student_id)` returns `0`. `COUNT(*)` would have falsely returned `1`.

3.3 Grouping Semantics & The `ONLY_FULL_GROUP_BY` Standard

MySQL 8.0 strictly enforces the SQL-92 / SQL:1999 standard `ONLY_FULL_GROUP_BY`.

Under this rule, any column appearing in the `SELECT` list that is **not** enclosed in an aggregate function **must be explicitly present in the `GROUP BY` clause**, unless the column is functionally dependent on the primary key in the `GROUP BY`.

For example, in Query `Q05`:

```
SELECT d.name, COUNT(s.student_id) AS student_count
FROM department AS d
JOIN student AS s ON s.department_id = d.department_id
GROUP BY d.department_id, d.name
HAVING COUNT(s.student_id) >= 5;
```

Both `d.department_id` and `d.name` are included in the `GROUP BY` to ensure deterministic projection.

3.4 Filter Lifecycle: `WHERE` vs `HAVING`

Understanding the query execution lifecycle is essential for DBMS viva defense:

```
[1. FROM & JOIN]  --> Assembles base Cartesian product & join conditions
        |
        ▼
[2. WHERE]        --> Row-by-row filter on BASE tuples BEFORE grouping
        |
        ▼
[3. GROUP BY]     --> Partitions remaining rows into aggregation buckets
        |
        ▼
[4. AGGREGATES]   --> Computes COUNT(), SUM(), AVG(), etc. per bucket
        |
        ▼
[5. HAVING]       --> Bucket-by-bucket filter on COMPUTED AGGREGATES
        |
        ▼
[6. WINDOW]       --> Computes analytic functions (DENSE_RANK(), ROW_NUMBER())
        |
        ▼
[7. SELECT]       --> Projects columns, evaluates expressions & aliases
        |
        ▼
[8. ORDER BY]     --> Sorts the final output stream
        |
        ▼
[9. LIMIT/OFFSET] --> Truncates the emitted result set
```

- `WHERE` **executes in Step 2:** It cannot reference aggregate functions like `COUNT(*)` because grouping has not occurred yet.
- `HAVING` **executes in Step 5:** It filters the summarized buckets generated by `GROUP BY`.

In Query `Q05`, we filter departments with at least 5 registered students:

- `WHERE` filters out inactive students if needed.
- `HAVING COUNT(s.student_id) >= 5` filters out departments whose total student bucket is smaller than 5.

3.5 Boolean Expression Aggregation (`SUM(condition)`)

In MySQL, boolean comparisons evaluate to `1` (True), `0` (False), or `NULL` (Unknown). We leverage this mathematical property to build high-performance pivot rollups without subqueries or procedural cursors.

In `v_medal_tally`:

```
SELECT
    department_id,
    department_name,
    SUM(award_type = 'gold') AS gold,
    SUM(award_type = 'silver') AS silver,
    SUM(award_type = 'bronze') AS bronze,
    COUNT(*) AS total_awards
FROM award_departments
GROUP BY department_id, department_name;
```

Execution Mechanics:

For each row in a department's award bucket:

- If `award_type` is `'gold'`, `(award_type = 'gold')` evaluates to `1`.
- If `award_type` is `'silver'`, it evaluates to `0`.
- Summing these integer representations yields the exact gold medal count in a single sequential pass over the data!

3.6 Type Safety: Unsigned Integer Underflow & `CAST(... AS SIGNED)`

In our physical database schema (`sql/schema.sql`), match scores are stored as:

```
home_score SMALLINT UNSIGNED NOT NULL DEFAULT 0,
away_score SMALLINT UNSIGNED NOT NULL DEFAULT 0
```

Under MySQL's default `NO_UNSIGNED_SUBTRACTION` SQL mode, subtracting two unsigned integers when the subtrahend exceeds the minuend (e.g., $1 - 3$) does **not** produce -2; it triggers a fatal runtime error:

`BIGINT UNSIGNED value is out of range`.

To calculate goal difference (`goals_for - goals_against`) safely, we explicitly cast both attributes to signed integers before subtraction:

```
COALESCE(SUM(
    CAST(tr.goals_for AS SIGNED) - CAST(tr.goals_against AS SIGNED)
), 0) AS goal_difference
```

This ensures that teams with negative goal differences (e.g. -4) are represented accurately without integer underflow exceptions.

3.7 Null Coalescing with `COALESCE()`

The ANSI SQL `COALESCE(val1, val2, ..., valN)` function returns the first non-`NULL` argument in its parameter list.

When a team has zero completed matches, the `LEFT JOIN` yields `NULL` for `SUM(tr.points)`. Presenting `NULL` to end users or application code can cause `NaN` display errors or broken table layouts.

Wrapping all aggregate sums in `COALESCE(SUM(...), 0)` guarantees that every team has a valid, non-null numeric score:

- Points: `0`
- Goal Difference: `0`
- Won / Drawn / Lost: `0`

---

4. Advanced Relational Constructs: CTEs & Window Functions

4.1 Common Table Expressions (`WITH ... AS`)

A Common Table Expression (CTE) defines a named, temporary result set that exists only within the scope of a single SQL statement.

Advantages of CTEs over Nested Subqueries:

1. **Readability and Modularity:** Decomposes complex multi-stage transformations into top-down, self-documenting computational steps.
2. **Reusability:** A CTE can be referenced multiple times within the same downstream query without duplicating SQL code.
3. **Optimizer Efficiency:** MySQL 8.0 can choose to either merge the CTE definition directly into the parent query plan or materialize it into an internal temporary table.

Our `v_points_table` view links three sequential CTEs:

1. `stage_teams`: Extracts all registered teams belonging to the stage.
2. `team_results`: Expands each match into two symmetrical perspectives (home and away).
3. `aggregated`: Joins the first two CTEs and computes mathematical totals.

4.2 Window Functions vs Standard Aggregation

Standard `GROUP BY` collapses multiple input rows into a single summary row. In contrast, **Window Functions** perform calculations across a set of table rows that are related to the current row, **while retaining the individual identity of each row in the output**.

The general syntax is:

$$\text{function}() \text{ OVER } (\text{PARTITION BY } col_1, \dots \text{ ORDER BY } col_2 \dots)$$

- `PARTITION BY`: Divides the query result set into independent processing partitions (similar to `GROUP BY`, but without collapsing rows).
- `ORDER BY`: Defines the logical sequence of rows within each partition for ranking or running totals.

4.3 Comparative Ranking: `DENSE_RANK()` vs `RANK()` vs `ROW_NUMBER()`

Function	Handling of Ties (Equal Order Values)	Sequence Gap Behavior	Example Output for Points (9, 7, 7, 4)
`ROW_NUMBER()`	Assigns arbitrary sequential integers; breaks ties arbitrarily.	No gaps (1, 2, 3, 4, ...)	1, 2, 3, 4
`RANK()`	Gives identical rank to tied rows.	**Leaves gaps** corresponding to the number of tied rows.	1, **2, 2, 4** (Rank 3 is skipped!)
`DENSE_RANK()`	Gives identical rank to tied rows.	**Never leaves gaps**; consecutive integers are always used.	1, **2, 2, 3** (Rank 3 is preserved!)

Why We Chose `DENSE_RANK()` for Tournament Standings:

In a sports tournament, if two teams are tied for 2nd place, the next team in line should be ranked 3rd (not 4th). `DENSE_RANK()` guarantees consecutive podium positions across all groups:

```
DENSE_RANK() OVER (
    PARTITION BY aggregated.event_id, aggregated.stage_id
    ORDER BY points DESC, goal_difference DESC, goals_for DESC, team_id
) AS points_position
```

Notice the tie-breaker hierarchy:

1. Primary sort: `points DESC` (most points wins)
2. Secondary sort: `goal_difference DESC` (better goal difference wins)
3. Tertiary sort: `goals_for DESC` (more goals scored wins)
4. Deterministic tie-breaker: `team_id ASC` (guarantees a stable, deterministic ordering)

---

5. Deep-Dive Walkthrough of Flagship Production Queries

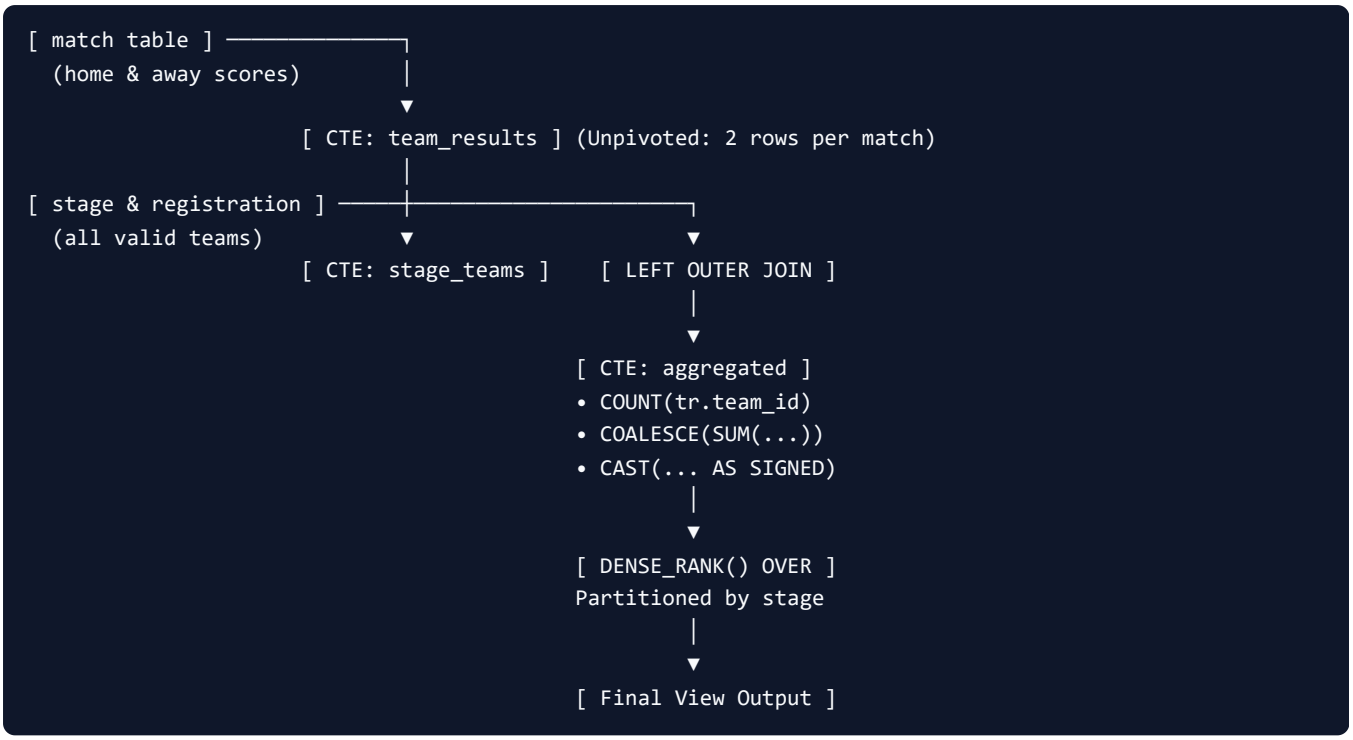
5.1 Deep Dive: `v_points_table` (Multi-CTE Standings Engine)

Located at: `sql/views.sql:10-111`

```
CREATE VIEW v_points_table AS
WITH stage_teams AS (
    -- Step 1: Identify all teams competing in this group stage.
    -- We union home teams from matches, away teams from matches,
    -- and registered teams for un-scheduled groups.
    SELECT DISTINCT s.event_id, s.stage_id, s.name AS group_name, m.home_team_id AS team_id
    FROM stage s
    JOIN `match` m ON m.stage_id = s.stage_id
    WHERE s.stage_type = 'group'
    UNION
    SELECT DISTINCT s.event_id, s.stage_id, s.name AS group_name, m.away_team_id AS team_id
    FROM stage s
    JOIN `match` m ON m.stage_id = s.stage_id
    WHERE s.stage_type = 'group'
    UNION
    SELECT DISTINCT s.event_id, s.stage_id, s.name AS group_name, r.team_id
    FROM stage s
    JOIN registration r ON r.event_id = s.event_id
    WHERE s.stage_type = 'group'
      AND r.team_id IS NOT NULL
      AND r.status = 'approved'
      AND NOT EXISTS (SELECT 1 FROM `match` m WHERE m.stage_id = s.stage_id)
),
team_results AS (
    -- Step 2: Unpivot match records so each match contributes two rows:
    -- one from the home team perspective and one from the away team perspective.
    SELECT
        s.event_id,
        m.stage_id,
        m.home_team_id AS team_id,
        m.home_score AS goals_for,
        m.away_score AS goals_against,
        CASE
            WHEN m.home_score > m.away_score THEN 3
            WHEN m.home_score = m.away_score THEN 1
            ELSE 0
        END AS points,
        CASE WHEN m.home_score > m.away_score THEN 1 ELSE 0 END AS wins,
        CASE WHEN m.home_score = m.away_score THEN 1 ELSE 0 END AS draws,
        CASE WHEN m.home_score < m.away_score THEN 1 ELSE 0 END AS losses
    FROM `match` AS m
    JOIN stage AS s ON s.stage_id = m.stage_id
    WHERE s.stage_type = 'group' AND m.status = 'completed'
    UNION ALL
    SELECT
        s.event_id,
        m.stage_id,
        m.away_team_id AS team_id,
        m.away_score AS goals_for,
        m.home_score AS goals_against,
        CASE
            WHEN m.away_score > m.home_score THEN 3
            WHEN m.away_score = m.home_score THEN 1
            ELSE 0
        END AS points,
        CASE WHEN m.away_score > m.home_score THEN 1 ELSE 0 END AS wins,
        CASE WHEN m.away_score = m.home_score THEN 1 ELSE 0 END AS draws,
        CASE WHEN m.away_score < m.home_score THEN 1 ELSE 0 END AS losses
    FROM `match` AS m
```

```
    JOIN stage AS s ON s.stage_id = m.stage_id
    WHERE s.stage_type = 'group' AND m.status = 'completed'
),
aggregated AS (
    -- Step 3: Outer-join registered teams with match results and compute summaries.
    SELECT
        st.event_id,
        st.stage_id,
        st.group_name,
        st.team_id,
        COUNT(tr.team_id) AS played,
        COALESCE(SUM(tr.wins), 0) AS won,
        COALESCE(SUM(tr.draws), 0) AS drawn,
        COALESCE(SUM(tr.losses), 0) AS lost,
        COALESCE(SUM(tr.goals_for), 0) AS goals_for,
        COALESCE(SUM(tr.goals_against), 0) AS goals_against,
        COALESCE(SUM(
            CAST(tr.goals_for AS SIGNED) - CAST(tr.goals_against AS SIGNED)
        ), 0) AS goal_difference,
        COALESCE(SUM(tr.points), 0) AS points
    FROM stage_teams st
    LEFT JOIN team_results tr
      ON tr.stage_id = st.stage_id AND tr.team_id = st.team_id
    GROUP BY st.event_id, st.stage_id, st.group_name, st.team_id
)
-- Step 4: Final join with descriptive parent tables and window ranking.
SELECT
    aggregated.event_id,
    aggregated.stage_id,
    aggregated.group_name,
    e.name AS event_name,
    sp.name AS sport_name,
    aggregated.team_id,
    team.name AS team_name,
    aggregated.played,
    aggregated.won,
    aggregated.drawn,
    aggregated.lost,
    aggregated.goals_for,
    aggregated.goals_against,
    aggregated.goal_difference,
    aggregated.points,
    DENSE_RANK() OVER (
        PARTITION BY aggregated.event_id, aggregated.stage_id
        ORDER BY points DESC, goal_difference DESC, goals_for DESC, team_id
    ) AS points_position
FROM aggregated
JOIN team ON team.team_id = aggregated.team_id
JOIN `event` AS e ON e.event_id = aggregated.event_id
JOIN sport AS sp ON sp.sport_id = e.sport_id;
```

Visual Transformation Flowchart:



5.2 Deep Dive: `v_medal_tally` (Polymorphic Exclusive-Arc Rollup)

Located at: `sql/views.sql:113-140`

```
CREATE VIEW v_medal_tally AS
WITH award_departments AS (
    SELECT
        d.department_id,
        d.name AS department_name,
        a.award_type
    FROM award AS a
    JOIN team AS t ON t.team_id = a.team_id
    JOIN department AS d ON d.department_id = t.department_id
    UNION ALL
    SELECT
        d.department_id,
        d.name AS department_name,
        a.award_type
    FROM award AS a
    JOIN student AS s ON s.student_id = a.student_id
    JOIN department AS d ON d.department_id = s.department_id
)
SELECT
    department_id,
    department_name,
    SUM(award_type = 'gold') AS gold,
    SUM(award_type = 'silver') AS silver,
    SUM(award_type = 'bronze') AS bronze,
    COUNT(*) AS total_awards
FROM award_departments
GROUP BY department_id, department_name;
```

Key Technical Mechanisms:

1. **Exclusive-Arc Normalization:** Resolves the polymorphism between individual student awards and squad team awards into a unified virtual stream `award_departments`.
2. **Boolean Expression Summing:** Avoids three separate subqueries by using `SUM(award_type = 'gold')`. In MySQL, if `award_type` is `'gold'`, the comparison yields `1`, which `SUM()` aggregates.
3. **Total Calculation:** `COUNT(*)` counts total awards received by each department across all categories.

5.3 Deep Dive: `v_match_schedule` (6-Table Operations Schedule)

Located at: `sql/views.sql:141-163`

```
CREATE VIEW v_match_schedule AS
SELECT
    m.match_id,
    e.event_id,
    e.name AS event_name,
    s.name AS stage_name,
    m.starts_at,
    m.duration_minutes,
    m.status,
    home.name AS home_team,
    away.name AS away_team,
    v.name AS venue_name,
    o.full_name AS official_name,
    o.role AS official_role
FROM `match` AS m
JOIN stage AS s ON s.stage_id = m.stage_id
JOIN `event` AS e ON e.event_id = s.event_id
JOIN team AS home ON home.team_id = m.home_team_id
JOIN team AS away ON away.team_id = m.away_team_id
JOIN venue AS v ON v.venue_id = m.venue_id
LEFT JOIN official AS o ON o.match_id = m.match_id
WHERE m.status IN ('scheduled', 'live');
```

Relational Path Breakdown:

1. `match` $\rightarrow$ `stage`: Retrieves the tournament bracket stage name.
2. `stage` $\rightarrow$ `event`: Traverses to the parent sport event.
3. `match` $\rightarrow$ `team (home)`: Translates `m.home_team_id` into the department squad name.
4. `match` $\rightarrow$ `team (away)`: Translates `m.away_team_id` into the opposing department squad name.
5. `match` $\rightarrow$ `venue`: Identifies the physical ground, court, or track.
6. `match` $\rightarrow$ `official` via **`LEFT JOIN`**: An official may not yet be assigned to a scheduled match. An `INNER JOIN` would cause unassigned matches to vanish from the calendar. The `LEFT JOIN` preserves the fixture while showing `NULL` for the referee name.

5.4 Deep Dive: `fn_team_points` (Deterministic Score Function)

Located at: `sql/procedures.sql:8-34`

```
CREATE FUNCTION fn_team_points(
    p_event_id INT UNSIGNED,
    p_team_id INT UNSIGNED
)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE v_points INT DEFAULT 0;

    SELECT COALESCE(SUM(
        CASE
            WHEN (m.home_team_id = p_team_id AND m.home_score > m.away_score)
              OR (m.away_team_id = p_team_id AND m.away_score > m.home_score)
                THEN 3
            WHEN m.home_score = m.away_score THEN 1
            ELSE 0
        END
    ), 0)
    INTO v_points
    FROM `match` AS m
    JOIN stage AS s ON s.stage_id = m.stage_id
    WHERE s.event_id = p_event_id
      AND m.status = 'completed'
      AND (m.home_team_id = p_team_id OR m.away_team_id = p_team_id);

    RETURN v_points;
END$$
```

Mechanics:

- Joins `match` with `stage` to filter by tournament event.
- Filters strictly on `m.status = 'completed'`.
- Uses a conditional `CASE` expression inside `SUM()`:
- 3 points if the specified team was home and scored more, OR was away and scored more.
- 1 point if the match was drawn.
- 0 points for a defeat.
- Returns `0` via `COALESCE` if the team has played no completed matches.

5.5 Deep Dive: `sp_generate_group_fixtures` & `sp_advance_knockout`

Located at: `sql/procedures.sql:38-176`

1. `sp_generate_group_fixtures` Window Ranking:

```
INSERT INTO fixture_teams (sequence_no, team_id)
SELECT
    ROW_NUMBER() OVER (ORDER BY r.team_id),
    r.team_id
FROM registration AS r
JOIN stage AS s ON s.event_id = r.event_id
WHERE s.stage_id = p_stage_id
  AND r.team_id IS NOT NULL
  AND r.status = 'approved'
GROUP BY r.team_id;
```

Uses `ROW_NUMBER() OVER (ORDER BY r.team_id)` to assign sequential indices $1, 2, \dots, N$ to all approved teams, enabling a clean round-robin nested Cartesian loop:

$$\forall i \in [1, N-1], \quad \forall j \in [i+1, N]: \quad \text{Schedule Match}(Team_i, Team_j)$$

2. `sp_advance_knockout` Window Partitioning:

```
INSERT INTO stage_winners (sequence_no, team_id)
SELECT
    ROW_NUMBER() OVER (ORDER BY group_name),
    team_id
FROM (
    SELECT
        group_name,
        team_id,
        ROW_NUMBER() OVER (
            PARTITION BY group_name
            ORDER BY points DESC, goal_difference DESC, goals_for DESC, team_id
        ) AS group_position
    FROM v_points_table
    WHERE stage_id = p_source_stage_id
) AS ranked_groups
WHERE group_position = 1;
```

- In the inner subquery, `ROW_NUMBER() OVER (PARTITION BY group_name ORDER BY ...)` assigns ranking positions inside each group independently.
- The outer query filters `WHERE group_position = 1` to harvest strictly the top seed from each group.
- The harvested winners are paired into knockout semi-final or final fixtures.

---

6. Complete Master Query Catalog Across the Project

Below is the exhaustive catalog of every query in the repository that uses Joins, Aggregates, CTEs, or Window Functions.

6.1 Database Views (`sql/views.sql`)

Object Name	Relational Operations Used	Joins Employed	Aggregates & Window Functions	Primary Purpose
`v_points_table`	CTE (`WITH`), `UNION` , `UNION ALL` , `LEFT JOIN` , `INNER JOIN`	`stage` $\bowtie$ `match` , `registration` $\bowtie$ `stage` , `stage_teams` $\mathbin{⋈}$ `team_results` , `team` , `event` , `sport`	`COUNT(tr.team_id)` , `SUM(wins)` , `SUM(draws)` , `SUM(losses)` , `SUM(goals_for)` , `SUM(goals_against)` , `SUM(CAST.. - CAST..)` , `SUM(points)` , `DENSE_RANK() OVER (...)`	Computes live round-robin standings, tie-breaking criteria, and consecutive positions per group.
`v_medal_tally`	CTE (`WITH`), `UNION ALL` , `INNER JOIN` , `GROUP BY`	`award` $\bowtie$ `team` $\bowtie$ `department` , `award` $\bowtie$ `student` $\bowtie$ `department`	`SUM(award_type = 'gold')` , `SUM(award_type = 'silver')` , `SUM(award_type = 'bronze')` , `COUNT(*)`	Resolves Exclusive-Arc awards into a single department leaderboard.
`v_match_schedule`	Multi-table `INNER JOIN` , `LEFT JOIN` , `WHERE IN`	`match` $\bowtie$ `stage` $\bowtie$ `event` $\bowtie$ `team (home)` $\bowtie$ `team (away)` $\bowtie$ `venue` $\mathbin{⋈}$ `official`	None (Projection over 6 tables)	Generates the operational tournament match schedule for live and upcoming fixtures.

6.2 Stored Routines (`sql/procedures.sql`)

Routine Name	Routine Type	Joins Employed	Aggregates & Window Functions	Primary Purpose
`fn_team_points`	Function	`match` $\bowtie$ `stage`	`SUM(CASE WHEN ... THEN 3 ... END)` , `COALESCE(...)`	Deterministically calculates cumulative league points for a specific team in an event.
`sp_generate_group_fixtures`	Procedure	`stage` $\bowtie$ `event` $\bowtie$ `sport` , `registration` $\bowtie$ `stage`	`ROW_NUMBER() OVER (ORDER BY r.team_id)` , `COUNT(*) INTO v_count`	Enumerates approved teams and inserts round-robin match pairings.
`sp_advance_knockout`	Procedure	Read from `v_points_table`	`ROW_NUMBER() OVER (PARTITION BY group_name ORDER BY ...)` , `ROW_NUMBER() OVER (ORDER BY group_name)`	Identifies group winners and generates subsequent knockout bracket fixtures.
`sp_assign_awards`	Procedure	Subquery on `match`	`CASE WHEN home_score > away_score ... END`	Automatically awards gold and silver medals to final match competitors.

6.3 Named Demonstration Queries (`sql/queries.sql`)

Query ID	Description	SQL Snippet Highlights	Joins Used	Aggregates / Analytical Constructs
Q01	List tournament events	`SELECT e.name, s.name FROM event e JOIN sport s ON s.sport_id = e.sport_id`	`INNER JOIN` (`event` , `sport`)	None
Q02	Upcoming schedule	`SELECT * FROM v_match_schedule ORDER BY starts_at`	Encapsulated in view	View query
Q03	Students by department	`SELECT s.full_name, d.name FROM student s JOIN department d ON d.department_id = s.department_id`	`INNER JOIN` (`student` , `department`)	None
Q04	Teams & squad sizes	`SELECT t.name, COUNT(tm.student_id) FROM team t LEFT JOIN team_member tm ... GROUP BY t.team_id, t.name`	`LEFT JOIN` (`team` , `team_member`)	`COUNT(tm.student_id)` , `GROUP BY`
Q05	Departments $\ge$ 5 students	`SELECT d.name, COUNT(s.student_id) FROM department d JOIN student s ... GROUP BY d.department_id HAVING COUNT(*) >= 5`	`INNER JOIN` (`department` , `student`)	`COUNT(s.student_id)` , `GROUP BY` , `HAVING`
Q06	Football registered teams	`WHERE EXISTS (SELECT 1 FROM registration r JOIN event e ON e.event_id = r.event_id ...)`	Correlated semi-join with `EXISTS`	Semi-join predicate
Q07	Students not in athletics	`WHERE NOT EXISTS (SELECT 1 FROM registration r JOIN sport sp ...)`	Correlated anti-join with `NOT EXISTS`	Anti-join predicate
Q08	Captains $\cup$ Athletes	`SELECT ... FROM team JOIN student ... UNION SELECT ... FROM registration JOIN student`	`INNER JOIN` in both branches	`UNION` (Set union with de-duplication)
Q09	Points table standings	`SELECT * FROM v_points_table ORDER BY points_position`	Encapsulated in view	`DENSE_RANK() OVER (...)`
Q10	High-scoring teams	`SELECT team_name, points FROM v_points_table WHERE points > 3`	View filter	View filter

Q11	Department medal tally	`SELECT * FROM v_medal_tally ORDER BY gold DESC, silver DESC`	Encapsulated in view	`SUM(award_type = 'gold')`
Q12	Athletics heat results	`SELECT e.name, s.full_name, ir.result_value FROM individual_result ir JOIN registration r ... JOIN student s ... JOIN event e`	4-table `INNER JOIN` (`individual_result` , `registration` , `student` , `event`)	None
Q14	Match victory margins	`SELECT ABS(CAST(home_score AS SIGNED) - CAST(away_score AS SIGNED)) AS margin FROM match JOIN team home ... JOIN team away`	Dual `INNER JOIN` to `team`	`ABS(CAST(.. AS SIGNED) - CAST(.. AS SIGNED))`
Q15	Goal scorers	`SELECT DISTINCT s.full_name, d.name FROM match_event me JOIN student s ... JOIN department d WHERE event_type = 'goal'`	3-table `INNER JOIN` (`match_event` , `student` , `department`)	`DISTINCT`
Q16	Routine score lookup	`SELECT fn_team_points(1, 1)`	Stored Function	Encapsulated conditional `SUM()`
Q17	Match officials activity	`SELECT o.full_name, COUNT(o.match_id) FROM official o GROUP BY o.official_id`	Single table projection	`COUNT(o.match_id)` , `GROUP BY`
Q18	Registrations by status	`SELECT e.name, r.status, COUNT(*) FROM registration r JOIN event e GROUP BY e.event_id, r.status`	`INNER JOIN` (`registration` , `event`)	`COUNT(*)` , Multi-column `GROUP BY`

6.4 Integrity Triggers (`sql/triggers.sql`)

`trg_check_eligibility_insert`	`BEFORE INSERT ON registration`	`event` **$\mathbin{⋈}$** `eligibility_rule` **$\bowtie$** `student`	**Verifies age, academic year, and gender constraints before registering an entrant.**
`trg_check_eligibility_update`	`BEFORE UPDATE ON registration`	`event` $\bowtie$ `student` $\mathbin{⋈}$ `eligibility_rule`	Re-verifies gender eligibility if participant or event references change.
`trg_venue_clash_insert`	`BEFORE INSERT ON match`	Self-comparison with `match`	Checks temporal interval intersection: `starts_at < new_end AND end > new_starts_at` .
`trg_venue_clash_update`	`BEFORE UPDATE ON match`	Self-comparison with `match`	Guarantees rescheduled matches do not conflict with existing bookings.
`trg_update_score`	`AFTER INSERT ON match_event`	Scalar subquery on `match`	Increments `home_score` or `away_score` when a `'goal'` event is logged.

6.5 Node.js Application Layer Queries (`app/server.js`)

`GET /events`	**Master Event Hub**	`event` **$\bowtie$** `sport` **$\bowtie$** `tournament`	**Scalar subqueries:** `COUNT(DISTINCT stage_id)`, `COUNT() ... status = 'completed'`, `COUNT( ) ... status = 'live'`, `COUNT(*) ... status = 'scheduled'`
`GET /events/:id`	Event Detail Page	`stage` $\mathbin{⋈}$ `match`, `match` $\bowtie$ `stage` $\bowtie$ `home` $\bowtie$ `away` $\bowtie$ `venue`, `registration` $\bowtie$ `student` $\bowtie$ `department`	`COUNT(m.match_id)` with `GROUP BY st.stage_id, st.name`
`GET /points`	Standings Page	`event` $\bowtie$ `sport` $\bowtie$ `stage`, read from `v_points_table`	Synchronizes empty group brackets with live calculated points standings
`GET /coordinator`	Match Operations Dashboard	5-table join: `match` $\bowtie$ `stage` $\bowtie$ `event` $\bowtie$ `sport` $\bowtie$ `home` $\bowtie$ `away`	Subquery counts for completed and scheduled stages
`POST /coordinator/advance`	Knockout Progression	Invokes `CALL sp_advance_knockout(?, ?, ?, ?)`	Executes windowed group advancement procedure
`POST /coordinator/awards`	Podium Generator	`stage` $\bowtie$ `event`, transactional query on `match`	Identifies 1st/2nd place winners from final match

---

7. DBMS Lab Viva Defense & Oral Examination Cheat-Sheet

Here are the 10 most common questions examiners ask about Joins and Aggregates, with model answers tailored to this project.

Q1: "Why did you use a `LEFT JOIN` in your points table instead of an `INNER JOIN`?"

> **Model Answer:**

> An `INNER JOIN` requires matching records in both tables. When a tournament stage is newly created, no matches have been played yet. If we used an `INNER JOIN` between teams and matches, all newly registered teams would be discarded and the points table would be completely empty!

> By using a `LEFT JOIN` from `stage_teams` to `team_results`, we guarantee that every registered team appears on the board immediately. For teams with no matches, match columns are `NULL`, which we turn into `0` using `COUNT(tr.team_id)` and `COALESCE(SUM(...), 0)`.

Q2: "What is the difference between `COUNT(*)` and `COUNT(column)`?"

> **Model Answer:**

> `COUNT(*)` counts the total number of rows in a partition, regardless of whether columns contain `NULL`. In contrast, `COUNT(column)` evaluates that specific column and ignores any `NULL` values.

> In our points table, when a team has played zero matches, the `LEFT JOIN` produces a row with `NULL` for `tr.team_id`. If we wrote `COUNT(*)`, it would count the row and falsely report 1 match played. By writing `COUNT(tr.team_id)`, MySQL ignores the `NULL` and correctly returns 0.

Q3: "What is the difference between `WHERE` and `HAVING`?"

> **Model Answer:**

> In the SQL execution lifecycle, `WHERE` executes *before* grouping occurs and filters individual rows. `HAVING` executes *after* `GROUP BY` and filters the aggregated groups based on summary functions.

> For example, in Query `Q05`, we use `HAVING COUNT(s.student_id) >= 5` to filter department groups after student counts have been calculated. A `WHERE` clause cannot do this because aggregates do not exist prior to grouping.

Q4: "Why did you choose `DENSE_RANK()` over `RANK()` and `ROW_NUMBER()` for standings?"

> **Model Answer:**

> `ROW_NUMBER()` assigns strictly distinct integers and would arbitrarily break ties between teams with identical records. `RANK()` leaves gaps in the ranking sequence (for example, if two teams tie for 1st, the next team is assigned rank 3).

> `DENSE_RANK()` assigns identical ranks to tied teams without skipping numbers (1st, 1st, 2nd, 3rd), which matches official sports tournament podium rules.

Q5: "How does `SUM(award_type = 'gold')` work without a `CASE` statement?"

> **Model Answer:**

> In MySQL, boolean conditions evaluate to integer `1` when true and `0` when false. In `v_medal_tally`, when a department's award is `'gold'`, `(award_type = 'gold')` evaluates to `1`. Summing these values calculates the exact gold medal tally in a single pass without needing three separate queries or complex joins.

Q6: "Why do you cast goal attributes to `SIGNED` before subtraction?"

> **Model Answer:**

> In our schema, `home_score` and `away_score` are declared as `SMALLINT UNSIGNED` to prevent negative scores. However, under MySQL's strict SQL mode, subtracting an unsigned integer from a smaller unsigned integer (e.g., $1 - 4$) causes a fatal out-of-range arithmetic overflow.

> We use `CAST(tr.goals_for AS SIGNED) - CAST(tr.goals_against AS SIGNED)` to ensure the difference can be negative (e.g., -3) for teams that concede more goals than they score.

Q7: "What is a Common Table Expression (CTE) and why use it over subqueries?"

> **Model Answer:**

> A CTE is a temporary, named result set defined with `WITH ... AS`. We use CTEs in `v_points_table` and `v_medal_tally` because they make complex multi-step queries modular and readable. In `v_points_table`, we separate seed team gathering, match result unpivoting, and final mathematical aggregation into three clean steps. CTEs also allow MySQL's optimizer to materialize or inline intermediate steps efficiently.

Q8: "How does MySQL execute joins internally under InnoDB?"

> **Model Answer:**

> When foreign key and primary key columns are indexed, MySQL uses an **Index Nested-Loop Join (INLJ)**. For each row in the driving table, it performs an $O(\log N)$ B-tree index lookup on the joined table. If no index is available for an equi-join, MySQL 8.0 uses an in-memory **Hash Join**, building a hash table on the smaller relation and probing it with the larger relation in $O(M+N)$ time.

Q9: "Why does the `match` table join the `team` table twice in `v_match_schedule`?"

> **Model Answer:**

> A match involves two distinct teams from the same entity set: a home team and an away team. The `match` relation has two foreign keys: `home_team_id` and `away_team_id`. To display the names of both teams in a single row, we must join `team` twice using table aliases (`home` and `away`).

Q10: "How did you solve the Exclusive ARC relationship in awards?"

> **Model Answer:**

> An award can belong to either a team or an individual student, enforced by the check constraint `(student_id IS NOT NULL) <> (team_id IS NOT NULL)`. In `v_medal_tally`, we used a CTE with `UNION ALL` to merge two join paths: one joining `award` $\rightarrow$ `team` $\rightarrow$ `department`, and another joining `award` $\rightarrow$ `student` $\rightarrow$ `department`. This projects a unified `(department_id, award_type)` relation that is then grouped and aggregated.

End of DBMS Queries, Joins, and Aggregates Comprehensive Guide.